

(VERY EARLY DRAFT ONLY)

Title: Contributions to Entity Component System and Scripting in Game Engines: A Practical Approach to Core Mechanics

João Henrique Schmidt de Carvalho - 119050097

Advisor: Geraldo Xexeo

Abstract

This work explores the integration of Entity Component System (ECS) and scripting into game engine development, with a focus on their roles in enhancing performance, maintainability, and modularity. ECS provides a data-driven approach to managing game entities and behaviors, addressing challenges related to memory locality, polymorphism overhead, and parallelism. Scripting facilitates dynamic interaction with the engine, allowing rapid iteration and greater accessibility when the ECS is unnecessary. This study analyzes their impact on core functionalities like graphics rendering and physics simulation. Key contributions include performance benchmarks, architectural insights, and practical guidelines for integrating ECS and scripting into game engines, with implications extending to simulation software and other performance-critical applications. This research aims to establish a robust foundation for future developments in modular and efficient game engine design.

1. Introduction Scheme:

The creation of game engines is a complex task that involves integrating multiple systems, including graphics, physics, input handling, sound, just to name a few. One of the central architectural designs of most modern game engines is the Entity Component System (ECS), which provides a flexible, data-driven approach to manage game entities and behaviors. Alongside ECS, scripting plays a pivotal role in enabling dynamic interaction with the engine with interpreted languages, such as Python, Lua, among others. Both ECS and scripting allow for modular, maintainable, and scalable engine design, which is essential for handling the complexity of modern games.

This TCC focuses on how these systems are built and their relationship with other engine components, mainly graphics and physics. The goal is to design and implement these systems

within a game engine, providing a deep understanding of their impact on performance, maintainability, and extensibility. Additionally, this work will explore how ECS and scripting influence other core systems, such as graphics rendering and physics simulations, demonstrating their importance in the overall architecture of game engines.

The significance of this research lies in the growing need for modular and efficient game engines. ECS has become a widely adopted pattern in the industry due to its ability to handle large-scale projects by decoupling data from logic, id est, the entity-component from systems.

Scripting, meanwhile, allows game developers to quickly iterate on gameplay without recompiling the entire codebase; it also makes it easier to apply further modifications by non-core developers. Understanding how these systems interact with graphics and physics can provide valuable insights for improving engine performance and flexibility. This knowledge is not only applicable to game development but can also be relevant for simulation software and other fields where rendering and physics are crucial.

First, I will talk about references and other essential tools used when building the ECS for a game engine or other big c-based projects in general. Then I will research and define the core architecture of ECS, identifying how entities, components, and systems will interact within the engine. This includes designing the data structures and memory layout. Next, I will implement the ECS and the registry, the latter will be useful to manipulate the complex data structures demonstrating how they are important to achieve the rendering and physics simulations. Finally, I will conduct performance benchmarks and tests to evaluate the efficiency and scalability of the ECS in handling big scenes, showing some examples of its interaction with other engine systems.

By the end of this project, I expect to have contributed a functional ECS and scripting system within Le Pain Engine which is a game engine I'm building as a hobby, with documented analysis of how these systems improve or challenge existing architectures in areas such as performance, scalability, and ease of use. This TCC will serve as a practical guide for developers looking to integrate ECS and scripting into their own projects, offering insights into the design choices, trade-offs, and challenges associated with these core systems.

2. Important Theoretical frameworks

Computer Systems: A Programmer's Perspective by Randal Bryant is sometimes called "The Computer Science Book" by some of my colleagues. This reference is key for understanding the lower-level aspects of computer systems, which are critical when building high-performance and clean software. It covers topics like memory management, CPU architecture, and efficient code execution, all of which are relevant to optimizing scripting systems, ECS and their data structures.

[[1]][1]

The detailed exploration of how systems interact with hardware will help me make informed decisions when managing resources in Pain Engine, ensuring that it remains both efficient and scalable. This book will be especially helpful in addressing challenges related to performance bottlenecks, as it provides insights into how software can be optimized at a deeper level.

Another aspect of this book is that it covers some unintuitive and invisible behaviors from the C programming language that can lead to bugs, even for advanced programmers. By doing so, it justifies some of the design choices that I'm using in my game engine.

The EnTT source code by [skypjack](#). EnTT is a well-established and highly efficient C++ library for implementing an Entity Component System (ECS). I chose to reference EnTT because it provides a robust and scalable ECS architecture that has been widely adopted in both hobbyist and professional game developments, like Minecraft, Diablo II, Call of Duty Vanguard, etc. Studying EnTT's source code will expose efficient ways to create functions to handle entities and components, especially in terms of memory management and performance optimizations. [\[\[2\]\]](#)[\[\[2\]\]](#)

Its use of modern C++ techniques, like template meta-programming, will serve as a valuable guide for implementing similar strategies in my own Registry system. The library is also made with the intent of being very simple to use and to modify if necessary, which is something Game Engine Architecture recommends when designing data-driven architectures.

ECST is an experimental multithreaded compile-time ECS library. It was developed as a Computer Science graduation project. I chose to reference ECST for its interesting approach to compile-time and multithreaded designs. Its focus on maximizing performance through parallel execution will be crucial for understanding how to handle large numbers of entities and components in complex game worlds. The library was made using C++14, but by studying ECST's source code, one could gain knowledge to design an ECS that leverages modern C++20 architectures. [\[\[3\]\]](#)[\[\[3\]\]](#)

3.1 Description of the problem:

There is no way around this subject other than to describe exactly what is being built here. As with any work, its purpose is to solve a problem, or at least to solve it in a better way than the other solutions.

Suppose that a developer is creating his/her game, and it needs to update millions of objects. Some of them are quite different from each other, so you have different structs of data.

For example, it might have objects like lights, meshes, audios, sprites, transforms, etc.

```
class Mesh extends Entity
class Light extends Entity
class Audio extends Entity
class Sprite extends Entity
class Transform extends Entity
```

Most probably, the object will be a combination of those above:

```
class MeshWithAudio extends Entity {
    Mesh *m_mesh
    Audio *m_audio
}
```

```

class LightWithAudio extends Entity {
    Light *m_mesh
    Audio *m_audio
}
class LightMeshWithAudio extends Entity {
    Light *m_light
    Mesh *m_mesh
    Audio *m_audio
}
class SpriteWithTransform extends Entity {
    Sprite *m_sprite
    Transform *m_transform
}

```

Or, if you are fond of inheritance, you could also use it:

```

class MeshWithAudio extends Mesh, Audio
class LightWithAudio extends Light, Audio
class LightMeshWithAudio extends Light, Mesh, Audio
class SpriteWithTransform extends Sprite, Transform

```

Either way, using those primary objects, you will then be able to create more concrete entities.

```

class Enemy extends MeshWithAudio
class Player extends MeshWithAudio
class Firefly extends LightWithAudio
class Lamp extends LightMeshWithAudio
class UID extends SpriteWithTransform

```

With that ready, the developer also needs to iterate between all those objects and update their data. Each object will have at least some sort of action in the game, for example, they need to be rendered, collide, process events, move, change states, etc.

Those direct approaches are prone to create two specific code styles:

```

For each Enemy
    call updatePhysics
    call checkCollisions
    call move
    call render
    // etc

```

```

For each Player
    call updatePhysics
    call checkCollisions
    call move
    call render
    // etc

```

```

For each Firefly

```

```
call move
call emit
// etc
```

However, this specific style, which is common in OOP, has three performance issues.

The first one is polymorphism overhead. Whenever you call a method of a base class or using interfaces, the call itself involves one indirect step before the actual code is reached, this is usually called Dynamic Dispatch Overhead and its cost usually depends on implementation. *C++* usually does this with the use of vTables, which are very straightforward indexed tables, which links virtual functions to real functions. But what is the overhead? At runtime the program needs to check inside the vTable what is the correct link between the virtual function and the actual function before performing the call. That extra step is what makes the program slow down^{[[6]][6]}

Figure 1: Example of vTable storing the linkage between two derived classes

The second problem is the memory locality. In the ECS case, the data will always be stored into a data structure of your choice, like an array or a map. Since you have the option to store all data in a contiguous section in your memory, there will never be a case where a different, less optimal data structure will be used instead, unless it is for a very exotic case such as if it strongly benefits from linked lists. However, such data structures can still be contiguously represented.

However, when iterating in the non ECS way, you are looping in the data of the entire object first before going to the next object. It is possible to clearly see the cost by doing a simple test. *Exempli gratia*, writing a small loop to sum all elements of a matrix:

```
Initialize result to 0
For each column index j from 0 to colsA - 1:
    For each row index i from 0 to rowsA - 1:
        Add the element A[i][j] to result
```

When implemented in C++, this code will execute in 4.46 seconds for a square matrix A of size 20000

```
Initialize result to 0
For each row index i from 0 to rowsA - 1:
    For each column index j from 0 to colsA - 1:
        Add the element A[i][j] to result
```

When implemented in C++, this code will execute in 1.15 seconds for a square matrix A of size 20000

What is happening here? In the memory, the data for matrix A is stored in the following layout:

```
A[0][0], A[0][1], A[0][2], ..., A[1][0], A[1][1], ...
```

When we iterate between all lines first, we are essentially jumping 20000 bytes of integers for every single integration. This leads to cache misses because each element is further apart in memory. In the end, this just shows what is already known, it is better for the CPU to iterate over arrays that are contiguous in memory.

With this we conclude the two disadvantages of not using an ECS style to deal with objects in a simulation.

However, there is a third, very indirect problem: which is the difficulty of implementing parallelism when compared to ECS...

To make a brief introduction as to why: the main problem with parallelization is finding units of processing which don't share any data, but because the data that is often iterated is mostly in the same set of components, there is a very high probability that the data inside a single for loop is unrelated to each other, which allow easy concurrency.

Because there isn't any concrete example of an ECS System is rather subjective at this stage, and can be shown better later when we discuss some implementations.

3.2.1 Explaining the ECS method: Components

With the main disadvantages of the direct approach already discussed, we can further investigate how the Entity Component System helps us to solve the main problem.

There isn't a bible of what makes a good ECS, because its purpose is to help with game development. So any explanation said here, although common when search through blogs and Q&A websites, are not strictly speaking *rules*. What is certain, is that it is performant enough to withstand different robustness tests, which are shown in the results section.

That being said, given the nature of the examples, it would be safe to assume that the implemented solution also has the capacity to withstand less robust, more common games, which is the case for most games on the market.

Like any other application, games have objects, like enemies, non-playable-characters, props, light sources, inventories, terrain, fluids, etc.

The first step is to turn a complex object into many small structs, such as: Mesh, Audio, Sprite, Transforms, Velocity, Angular Velocity, Collision Box, Particles, Camera, etc.

Those structures are called components, and they by themselves are just data without behavior. The idea is that they don't contain logic nor dependencies because the former is directive of the Systems and the latter is directive of the Archetypes.

3.2.2 Explaining the ECS method: Entity

There are different views for what constitutes an entity or what it's supposed to represent. For example, if you search on Wikipedia for ECS, they will define "Entity" as a general-purpose object, id est, a game object. Some articles at medium also define that way. In contrast, the Entity Systems Wiki defines as a container which components can be added. I, however, decided to go with the definition used by Unity Engine: Which states that entities are just indices which represent objects IDs. On C++ terms, it means that they are integers.

The reason to prefer this definition it's because it is simpler and because it works within Unity.

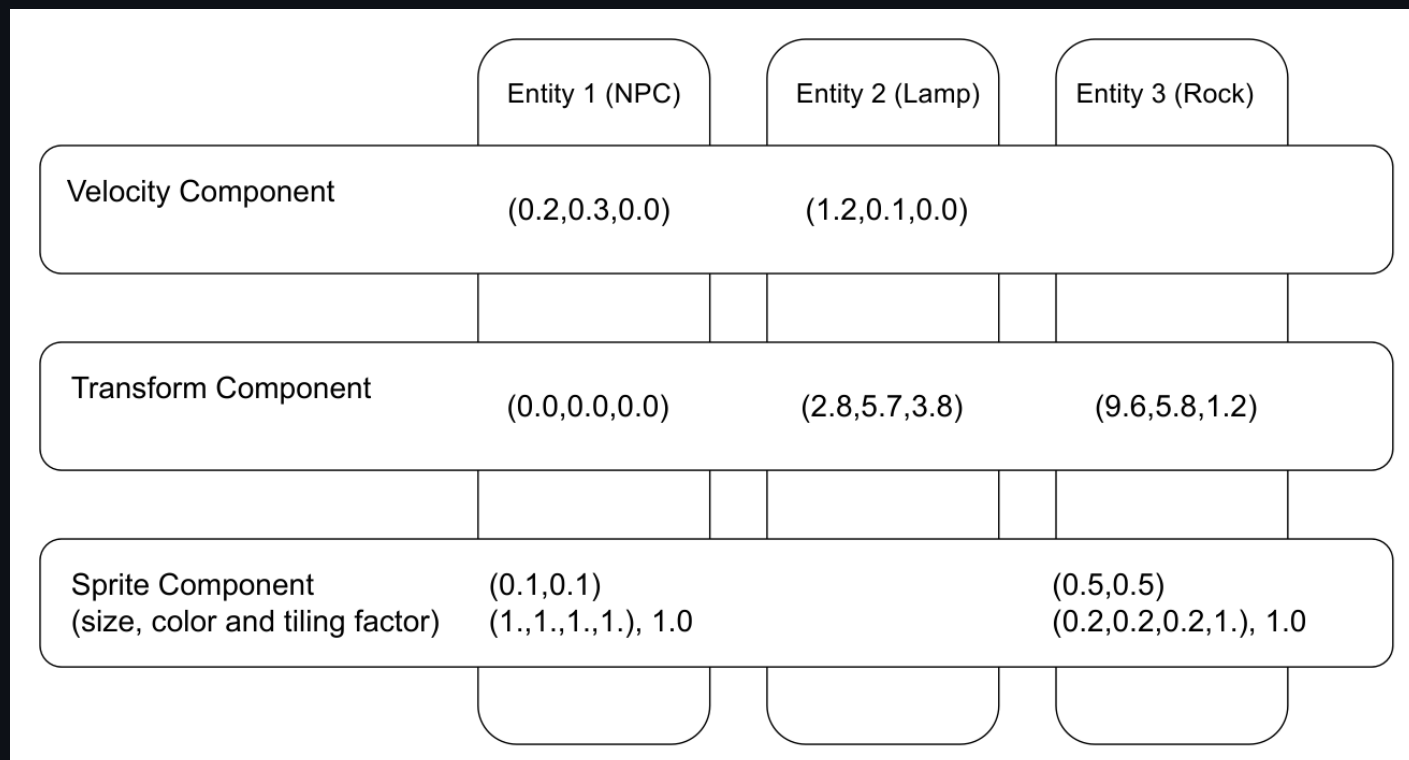


Figure 2: Arbitrary components from arbitrary entities filled with arbitrary data in a random order

As previously said, the data is actually stored inside data structures like vectors, maps, unordered maps, etc. Therefore, your objects can still be accessed using the entity ID.

For example, the NPC object can still be fully accessed by searching for its components (Velocity, Transform and Sprite) if you search for the index $ID = 1$:

```
call GetComponent velocity at index 1 // { (0.2,0.3,0.0) }
call GetComponent transform at index 1 // { (0.0,0.0,0.0) }
call GetComponent sprite at index 1 // { (0.1,0.1), (1.0,1.0,1.0,1.0), 1.0 }
```

Hence, what we now have is an extra layer of complexity inside our previous simple object. To call the entire object we would need to get every single component.

3.2.3 Explaining the ECS method: System

At first glance it might sound a bit weird to organize the data this way because we now have increased the cost of retrieving the original data. We now have to do N function calls, with N being the number of components in our object.

For small scale simulations where the number of objects is close to N , this is absolutely correct. However, we are not interested in small simulations because those tend to be performant already due to the small size.

To solve this issue, we must program the behavior separately from the data, using Systems. Systems are code that transform the data, each of them are functions that represent the logic.

To show this, let's consider the previous loop to update the enemy object:

```
For each Enemy
  call updatePhysics
  call checkCollisions
  call move
  call render
  // etc
```

Notice that some of those functions are really generic. In fact, they could be implemented elsewhere in a more abstract hierarchical way, by using inheritance. However, we don't want the overhead associated with polymorphism.

Therefore, let's see the ECS design for this problem by creating different Systems:

```
// physics system
For each Physics Component
  // update physics

// collisions system
For each Collision Component
  // check collisions

// movement system
For each Movement Component
  // move

// render system
For each Render Component
  // render
```

The behavior of each component is represented using a for loop that iterates over the entire data structure and performs the update. Using that, we solve both the memory locality, since the components are now close together in memory, and the polymorphism overhead, because each system function will be associated with a data structure.

As a rule of thumb, the code inside each loop is generally very specific, because its purpose is to update a very broad range of objects with a well-defined task. For example:

- A Physics System updates velocities based on gravity and other accelerations.
- A Movement System updates positions based on velocities.

- A Render System handles filling the batch rendering buffer
- A Collision System calculates the intersection between collision boxes

This also makes the loop very small compared to per object update. For example, the movement system here will have one line of code, making it easier for the compiler and for us to optimize the code.

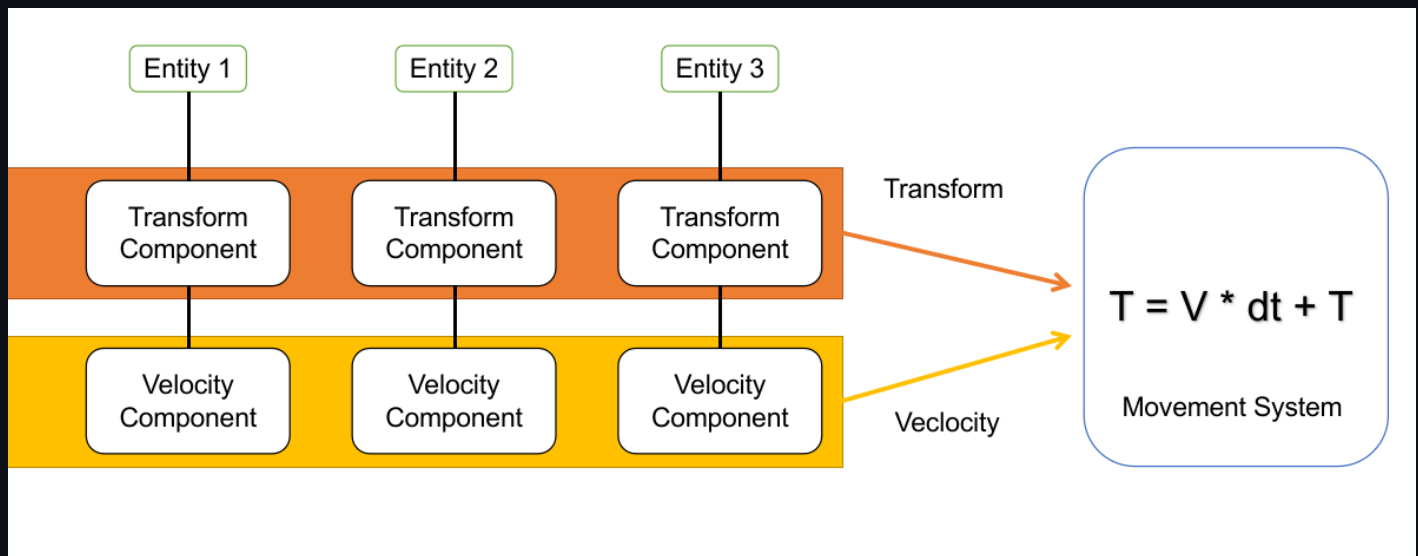


Figure 3: Movement System performs a scalar multiplication and a sum in two vectors

3.2.4 Explaining the ECS method: Registry

In programming, a wrapper is a program or code that surrounds other program components, providing an interface for easier interaction with the wrapped functionality.

Data structures have different implementations, but they need general functions to work with components and entities.

Going a little deeper inside the code, the registry serves as a general wrapper between the system functionality and data structures. They are responsible for:

- Create new entity IDs and recycle destroyed IDs.
- Retrieve the iterator which allow us to loop between components of the same type
- Emplace new components for a given ID
- Remove components for a given ID
- Check if component exist for a given ID
- Get specific component for a given ID
- Allow us to create a new data structure for a given component, preferably at compile time
- Allow us to remove a new data structure for a given component, preferably at compile time

Registries are hard because they will be accessed every time, making them very performance critical. Usually requiring different template specific code to be able to run as clean as possible, making the developer interaction with the registry feels as if it isn't even there.

3.2.5 Explaining the ECS method: Archetypes

With the registry being built we can now start to talk about Archetypes. Archetypes are sets of components that are joined together. They represent the dependency between components.

For example, if we want to loop between components of the velocity type, so we can update the Transform component, we obviously need them both. One possible solution would be to search for the second component:

```
for each Velocity Component
  search Transform Component
  Transform = Transform + Velocity * deltaTime
```

However, the search algorithm is costly.

Transform

Velocity

Sprite

Figure 4: Example of common components inside the registry, each square is an array

So a way to avoid this is to group the necessary components per system. If only the transform and the velocity system were in the same place in memory, they could be automatically brought together and be processed.

That's the job of the Archetypes. They work inside the registry and allow it to join both structs together. In the example, the Movement System will request both, and the registry will send an iterator with elements containing both.

This iterator will come from a special data structure that exists only for that purpose. Now, whenever an entity has both components, the registry can choose to place them inside it.

Those special data structures are the Archetypes. Internally they are just Structure of Arrays (SoA),

this ensures a contiguous memory access for all components within the same archetype.

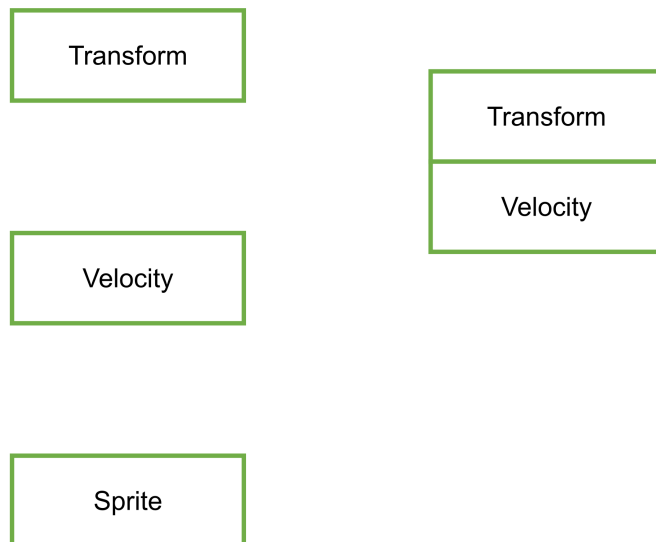


Figure 5: Example of common components inside the registry after the Movement System

What is happening here? The data for a specific System is being glued so that we can quickly iterate into all data that fits our specific component requirements.

With this we don't need to search for the necessary components. However, this also introduces some drawbacks.

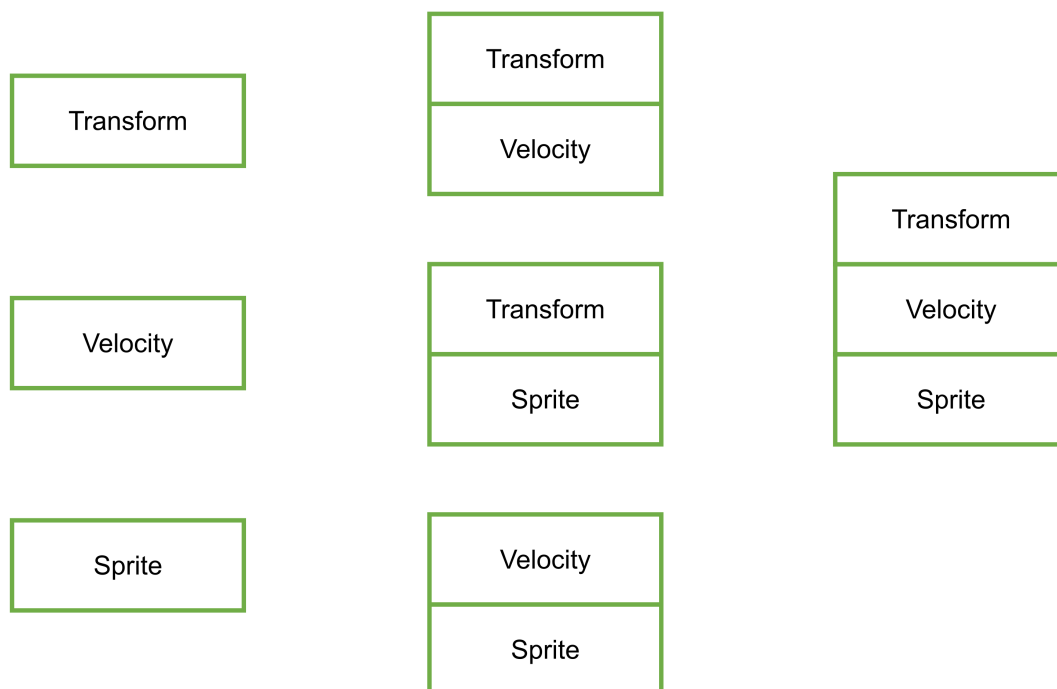


Figure 6: All possible combinations of components

Now, every time we want to just process one component, for example, the transform component, we must look for every Archetype that has said component and loop inside them.

The maximum possible number of archetypes is $2^N - 1$, where N is the number of components. But that is a bit of an exaggeration, and a number this high would actually compromise the loop instead of help.

In truth, the number of archetypes will depend on the number of systems that require them. Since they help the systems to perform better, if no behavior needs it, then it shouldn't exist.

For our problem, there is only two types of archetypes:

- Archetype 1: Transform Component, Velocity Component
- Archetype 2: Transform Component, Velocity Component, Sprite Component

Whenever the Movement System needs to update, it will call both archetypes.

However, whenever the Render System needs to update, it can call only Archetype 2 and receive the Sprite and Transform component arrays.

You all that being said, let's go back to the previous Movement System:

```
archetype = getArchetype(TransformComponent, VelocityComponent)
for each entity in archetype:
    entity.transform = entity.transform + (entity.velocity * deltaTime)
```

With this, we successfully iterate both components without an additional search overhead.

Notice that, for the Movement System, this design isn't free, because now its `for` loop will be effectively cut into two for it to correctly iterate both arrays.

3.2.6 Explaining the ECS method: Scripting

After reviewing the ECS, anyone would be thinking on how hard it would be to replace the direct approach with the ECS approach for all the problems. After all, to write everything you must abstract based on behaviors of all entities, and not based on individual objects.

This is where the part of scripting comes to help. For parts of the code that aren't critical, you can more safely use a slower and easier approach that abstracts many specifications of your engine. That way, game/mod developers can quickly script what they want.

This solution is often used together with a scripting language, which also abstract the C++ to a more simple and friendly language, such as Python, Lua, C#, JavaScript, etc.

3.3 Expected Results of Simulation:

To fully explore and validate the expected results, I design simulations and small games that stress-

test my game engine. These tests serve both to highlight its capabilities and to identify areas for optimization. Performance considerations will be emphasized, requiring scenarios where computational efficiency has a significant impact.

The expected outcomes include:

- A measurable increase in performance compared to baseline implementations.
- Clearly defined and modular systems, ensuring maintainability and extensibility.
- A scripting system that is performant when compared to traditional compiled languages like C# and C++.
- A robust registry system that matches or surpasses the performance of EnTT or similar ECS projects.

3.3.1 Planned Test Cases:

To achieve these results, I will implement and test the engine using the following scenarios:

- **AI Battle Simulation:** Create a clone of the Pezza [AI battle simulation](#), where multiple AI agents interact in real-time. This will stress-test the ECS, scripting, and real-time decision-making systems.
- **Physics Simulations:** Implement simulations involving water or numerous colliding particles, scenarios commonly found in physics-heavy games. This will validate the engine's physics system and its ability to handle numerous calculations efficiently.
- **High-Object Density Scene:** Design a small scene with a significant number of particles and objects, then intentionally push the system to its limits to identify breaking points and performance bottlenecks.
- Et cetera.

3.3.2 Achieving Robustness:

While game engines are never truly “finished,” the goal is to push the boundaries of what most general-purpose engines achieve within the defined scope of this project. By iterating on each of these tests and striving to exceed or match the benchmarks of existing engines, I aim to demonstrate the robustness and performance of the systems I have built. These efforts will also provide a foundation for future enhancements and expansions of the engine.

References

[1] R. E. Bryant and D. R. O'Hallaron, Computer systems: a programmer's perspective, 2. ed. Boston, Mass.: Prentice Hall, 2011.

[2] M. Caini, [skypjack/entt](https://github.com/skypjack/entt). (Oct. 23, 2024). C++. Accessed: Oct. 23, 2024. [Online]. Available: <https://github.com/skypjack/entt>

[3] V. Romeo, vittorioromeo/ecst. (Oct. 23, 2024). C++. Accessed: Oct. 23, 2024. [Online]. Available: <https://github.com/vittorioromeo/ecst>

[4] “Learn OpenGL, extensive tutorial resource for learning Modern OpenGL.” Accessed: Oct. 23, 2024. [Online]. Available: <https://learnopengl.com/>

[5] J. Gregory, Game engine architecture, Third edition. in An A.K. Peters book. Boca Raton London New York: CRC Press, Taylor & Francis Group, 2019.

[6] K. Driesen and U. Hölzle, “The direct cost of virtual function calls in C++,” SIGPLAN Not., vol. 31, no. 10, pp. 306–323, Oct. 1996, doi: 10.1145/236338.236369.